Mount Google Drive in Colab

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
PROJECT_PATH = "/content/drive/MyDrive/weather-trend-forecasting"
DATA_PATH = PROJECT_PATH + "/data/GlobalWeatherRepository.csv"
```

```
import os

os.makedirs(PROJECT_PATH + "/outputs/figures", exist_ok=True)
os.makedirs(PROJECT_PATH + "/outputs/models", exist_ok=True)
```

Install required libraries

```
!pip install pandas numpy matplotlib seaborn scikit-learn plotly xgboost joblib
```

Show hidden output

```
!pip install prophet
```

Show hidden output

Import libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor, VotingRegressor, IsolationForest

import joblib
import warnings
warnings.filterwarnings("ignore")
```

Load the dataset

```
df = pd.read_csv(DATA_PATH)

df.head()
```

Show hidden output

```
df.info()
```

Show hidden output

```
df.columns
```

Show hidden output

```
df.isnull().sum().sort_values(ascending=False)
```

Show hidden output

Convert last_updated to datetime

```
df["last_updated"] = pd.to_datetime(df["last_updated"], errors="coerce")

df = df.dropna(subset=["last_updated"])

df["year"] = df["last_updated"].dt.year
df["month"] = df["last_updated"].dt.month
df["day"] = df["last_updated"].dt.day
df["dayofweek"] = df["last_updated"].dt.dayofweek
```

Data cleaning and preprocessing

```
df = df.drop_duplicates()
```

```
numeric_cols = df.select_dtypes(include=["int64", "float64"]).columns
categorical_cols = df.select_dtypes(include=["object"]).columns
```

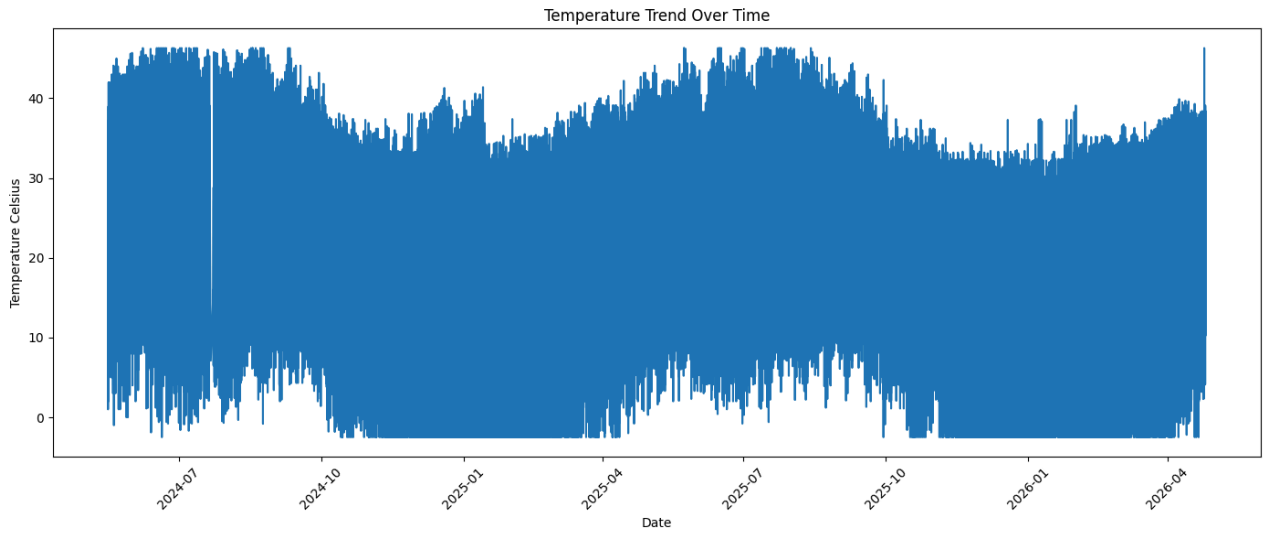
```
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median())
```

```
df[categorical_cols] = df[categorical_cols].fillna("Unknown")
```

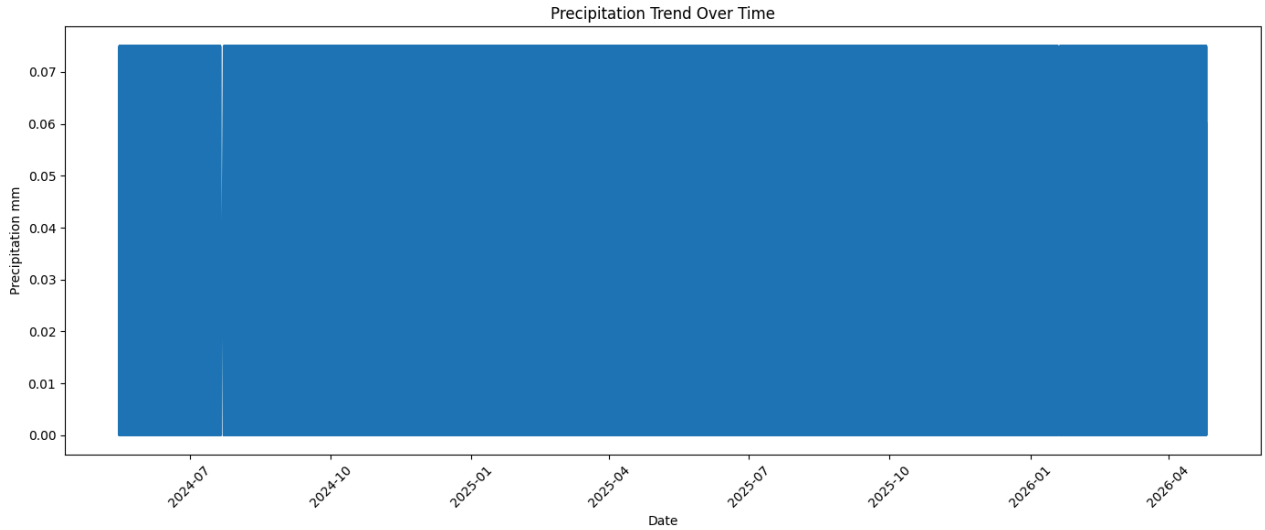
```
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df[col] = df[col].clip(lower, upper)
```

Basic EDA

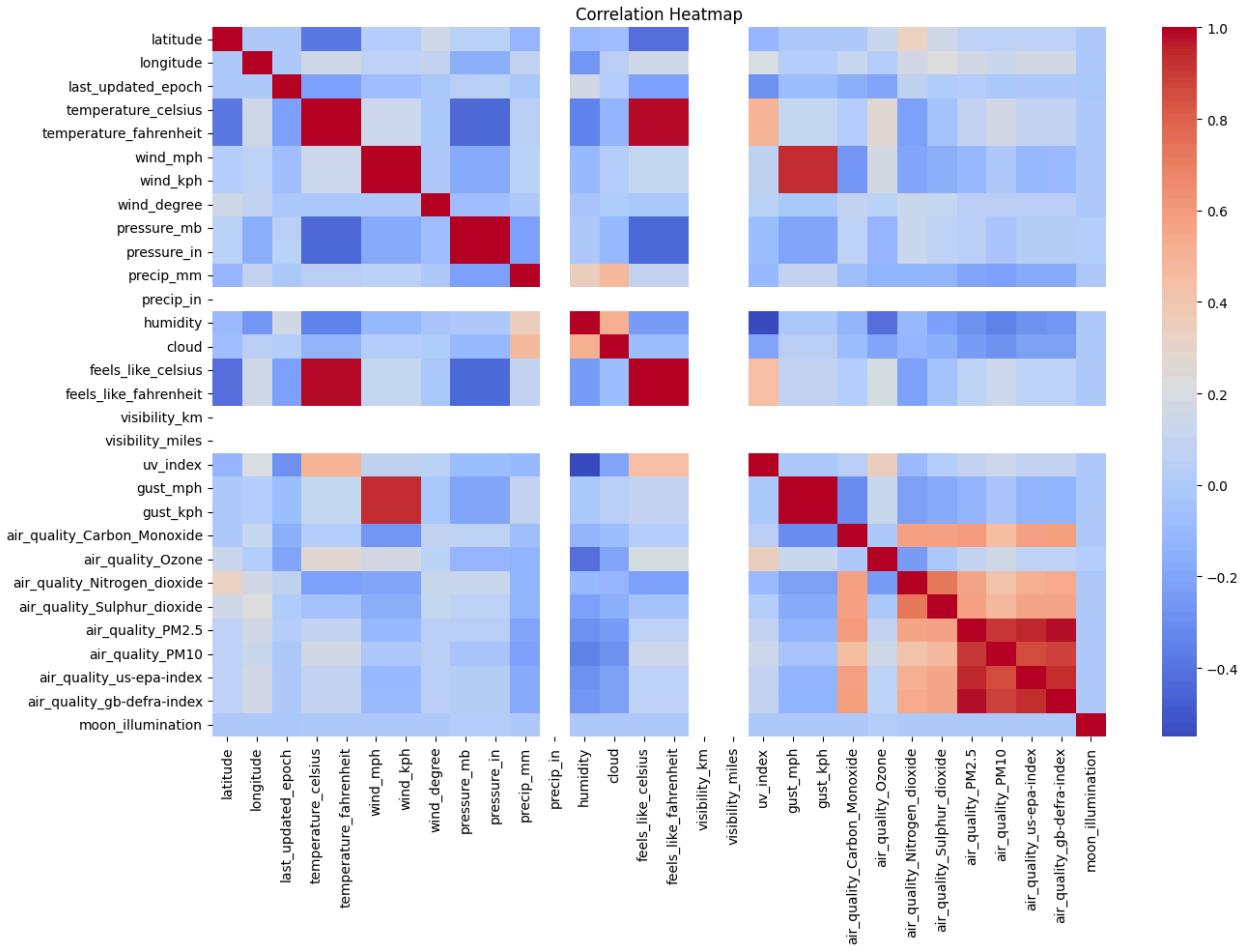
```
df_sorted = df.sort_values("last_updated")

plt.figure(figsize=(14, 6))
plt.plot(df_sorted["last_updated"], df_sorted["temperature_celsius"])
plt.title("Temperature Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Temperature Celsius")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/temperature_trend.png")
plt.show()
```




```
plt.figure(figsize=(14, 6))
plt.plot(df_sorted["last_updated"], df_sorted["precip_mm"])
plt.title("Precipitation Trend Over Time")
plt.xlabel("Date")
plt.ylabel("Precipitation mm")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/precipitation_trend.png")
plt.show()
```

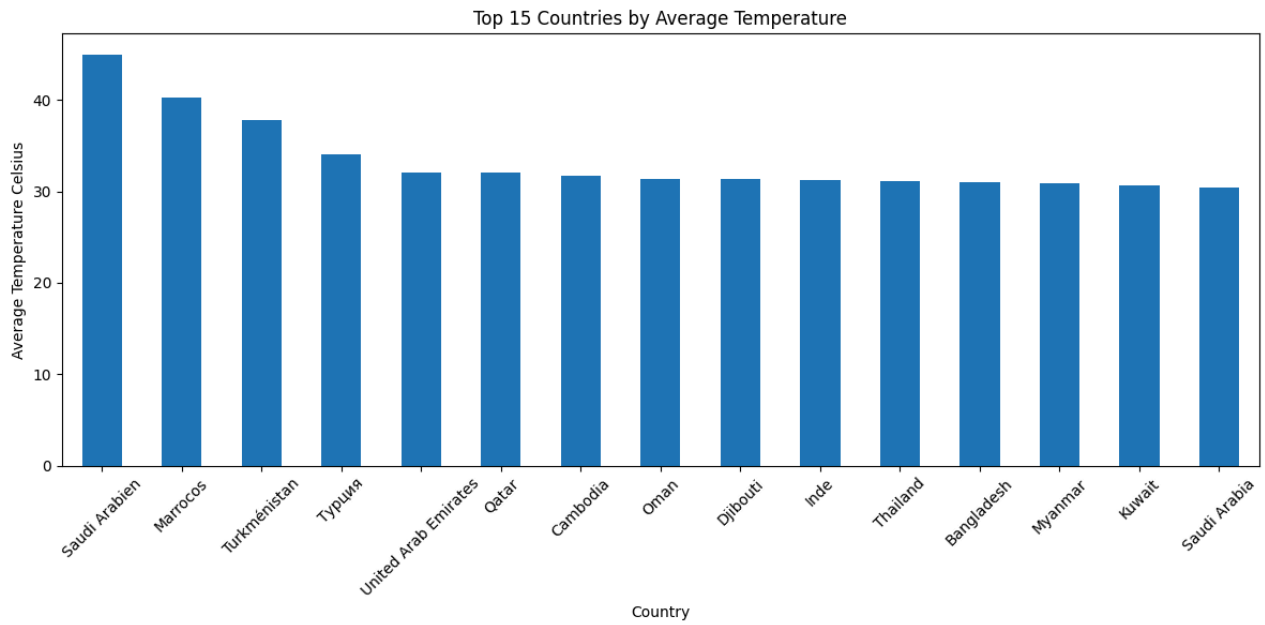



```
plt.figure(figsize=(14, 10))
sns.heatmap(df[numeric_cols].corr(), cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/correlation_heatmap.png")
plt.show()
```



Country-wise weather analysis

```
country_temp = df.groupby("country")["temperature_celsius"].mean().sort_values(ascending=False).head(15)

plt.figure(figsize=(12, 6))
country_temp.plot(kind="bar")
plt.title("Top 15 Countries by Average Temperature")
plt.xlabel("Country")
plt.ylabel("Average Temperature Celsius")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/top_country_temperature.png")
plt.show()
```



Build forecasting dataset

```
target = "temperature_celsius"

features = [
    "year",
    "month",
    "day",
    "dayofweek",
    "humidity",
    "wind_kph",
    "pressure_mb",
    "precip_mm"
]

available_features = [col for col in features if col in df.columns]

X = df[available_features]
y = df[target]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Build multiple forecasting models

```
models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(random_state=42),
    "Gradient Boosting": GradientBoostingRegressor(random_state=42)
}

results = []

for name, model in models.items():
    model.fit(X_train, y_train)
    preds = model.predict(X_test)

    mae = mean_absolute_error(y_test, preds)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)

    results.append({
```

```
        "Model": name,
        "MAE": mae,
        "RMSE": rmse,
        "R2 Score": r2
    })

results_df = pd.DataFrame(results)
results_df
```

	Model	MAE	RMSE	R2 Score
0	Linear Regression	6.063962	7.626486	0.351087
1	Random Forest	3.770463	5.376730	0.677467
2	Gradient Boosting	4.606714	6.012488	0.596684

```
results_df.to_csv(PROJECT_PATH + "/outputs/model_results.csv", index=False)
```

Create ensemble model

```
ensemble_model = VotingRegressor([
    ("linear", LinearRegression()),
    ("random_forest", RandomForestRegressor(random_state=42)),
    ("gradient_boosting", GradientBoostingRegressor(random_state=42))
])

ensemble_model.fit(X_train, y_train)
ensemble_preds = ensemble_model.predict(X_test)

ensemble_result = {
    "Model": "Voting Ensemble",
    "MAE": mean_absolute_error(y_test, ensemble_preds),
    "RMSE": np.sqrt(mean_squared_error(y_test, ensemble_preds)),
    "R2 Score": r2_score(y_test, ensemble_preds)
}

ensemble_result
```

```
{'Model': 'Voting Ensemble',
 'MAE': 4.589379025145863,
 'RMSE': np.float64(5.886700954303879),
 'R2 Score': 0.6133825272408048}
```

```
results_df = pd.concat(
    [results_df, pd.DataFrame([ensemble_result])],
    ignore_index=True
)

results_df
```

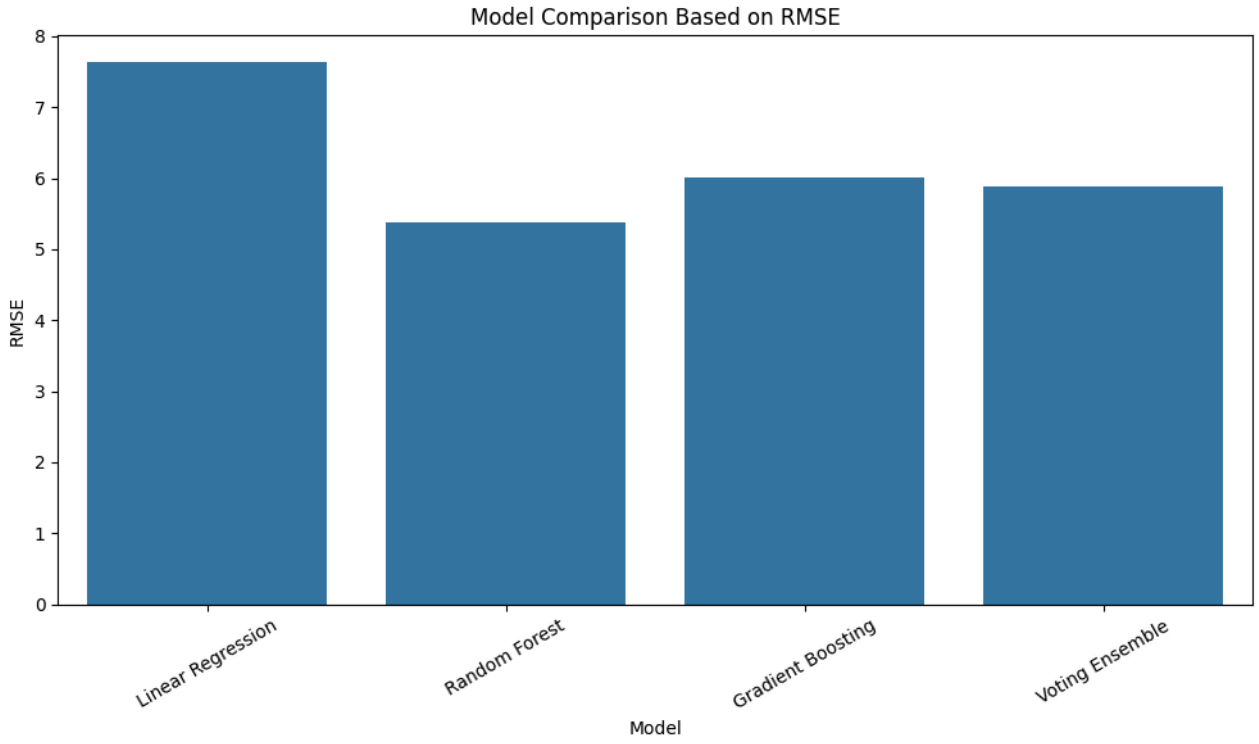
	Model	MAE	RMSE	R2 Score
0	Linear Regression	6.063962	7.626486	0.351087
1	Random Forest	3.770463	5.376730	0.677467
2	Gradient Boosting	4.606714	6.012488	0.596684
3	Voting Ensemble	4.589379	5.886701	0.613383

```
joblib.dump(ensemble_model, PROJECT_PATH + "/outputs/models/ensemble_weather_model.pkl")
```

Show hidden output

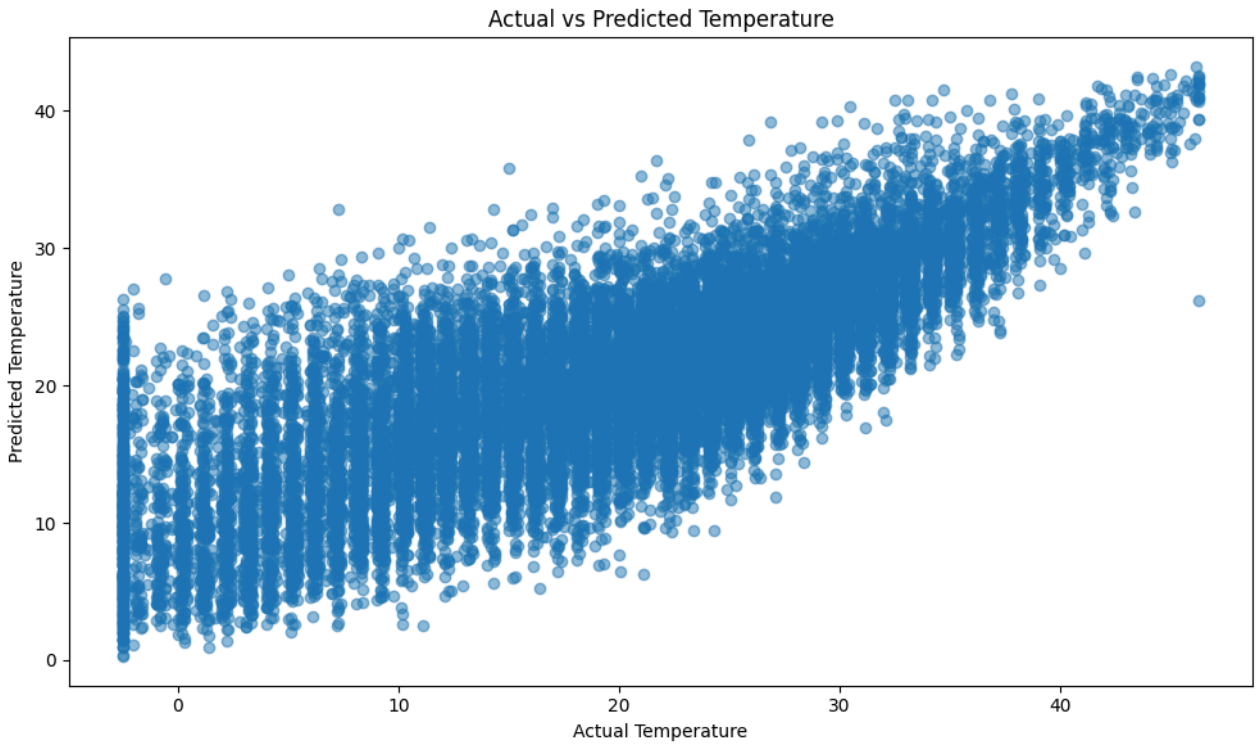
Visualize model comparison

```
plt.figure(figsize=(10, 6))
sns.barplot(data=results_df, x="Model", y="RMSE")
plt.title("Model Comparison Based on RMSE")
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/model_comparison_rmse.png")
plt.show()
```



Actual vs predicted graph

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, ensemble_preds, alpha=0.5)
plt.xlabel("Actual Temperature")
plt.ylabel("Predicted Temperature")
plt.title("Actual vs Predicted Temperature")
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/actual_vs_predicted.png")
plt.show()
```

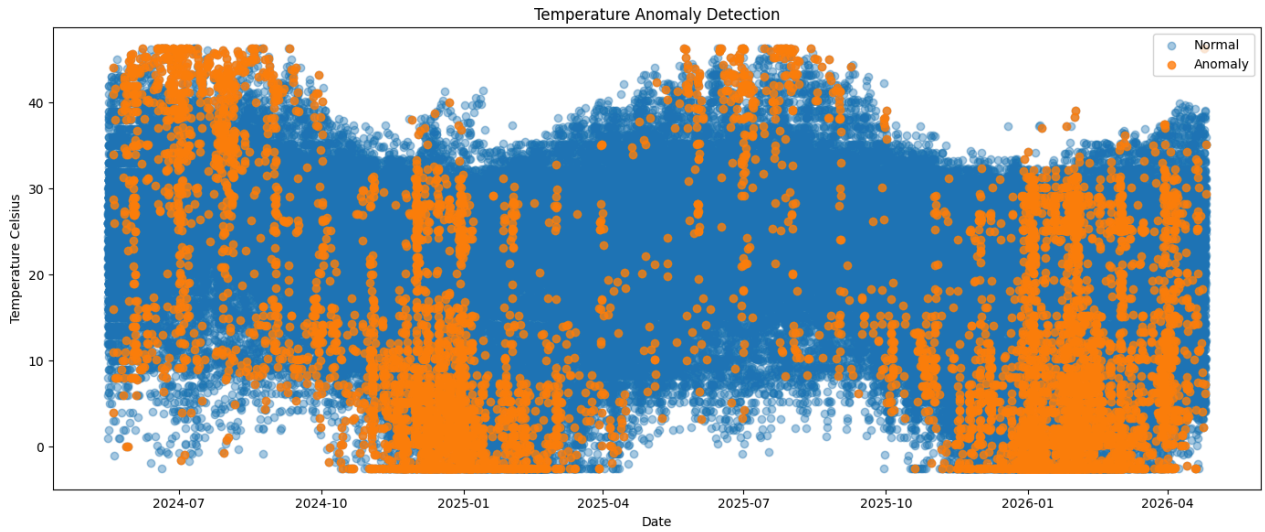


Advanced analysis — anomaly detection

```
anomaly_features = df[available_features + [target]]

anomaly_model = IsolationForest(contamination=0.05, random_state=42)
df["anomaly"] = anomaly_model.fit_predict(anomaly_features)

anomalies = df[df["anomaly"] == -1]

anomalies.head()
```

Show hidden output

```
plt.figure(figsize=(14, 6))
plt.scatter(df["last_updated"], df["temperature_celsius"], label="Normal", alpha=0.4)
plt.scatter(anomalies["last_updated"], anomalies["temperature_celsius"], label="Anomaly", alpha=0.8)
plt.title("Temperature Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Temperature Celsius")
plt.legend()
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/anomaly_detection.png")
plt.show()
```

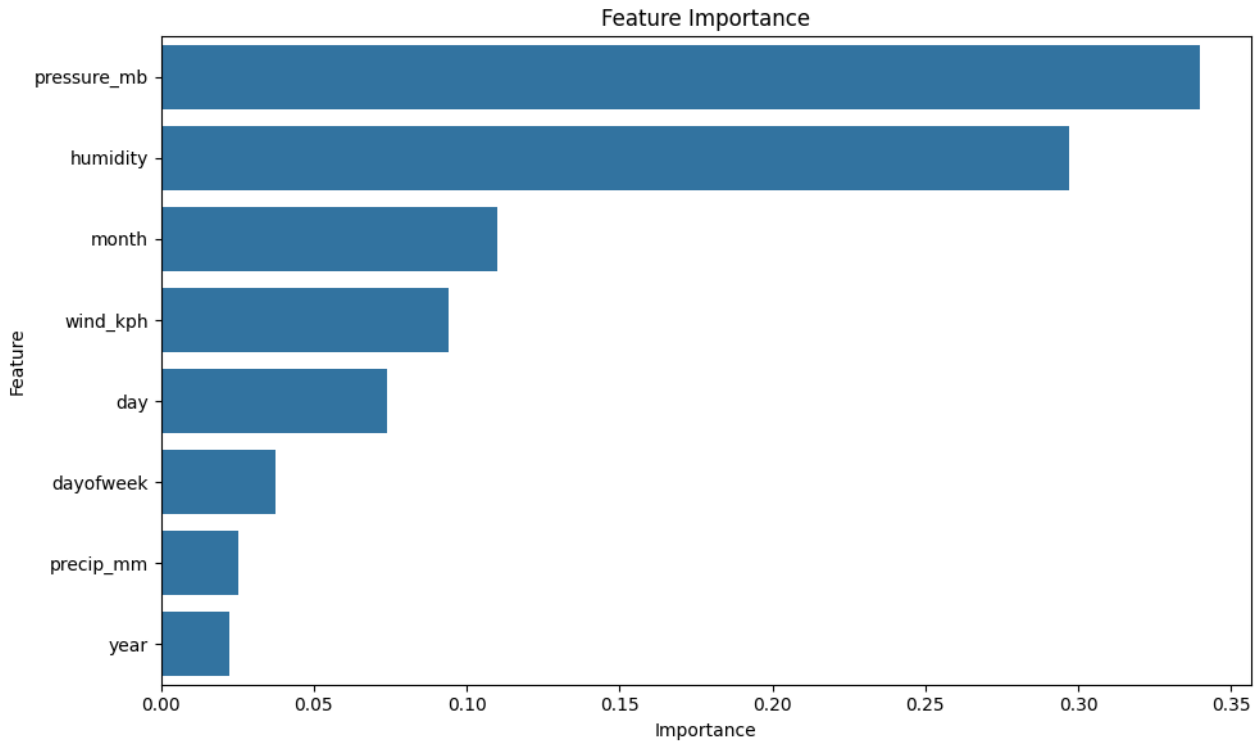


Advanced analysis — feature importance

```
rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

feature_importance = pd.DataFrame({
    "Feature": available_features,
    "Importance": rf_model.feature_importances_
}).sort_values(by="Importance", ascending=False)

feature_importance
```

	Feature	Importance
6	pressure_mb	0.339639
4	humidity	0.297047
1	month	0.110031
5	wind_kph	0.094290
2	day	0.073842
3	dayofweek	0.037617
7	precip_mm	0.025158
0	year	0.022377

```
plt.figure(figsize=(10, 6))
sns.barplot(data=feature_importance, x="Importance", y="Feature")
plt.title("Feature Importance")
plt.tight_layout()
plt.savefig(PROJECT_PATH + "/outputs/figures/feature_importance.png")
plt.show()
```



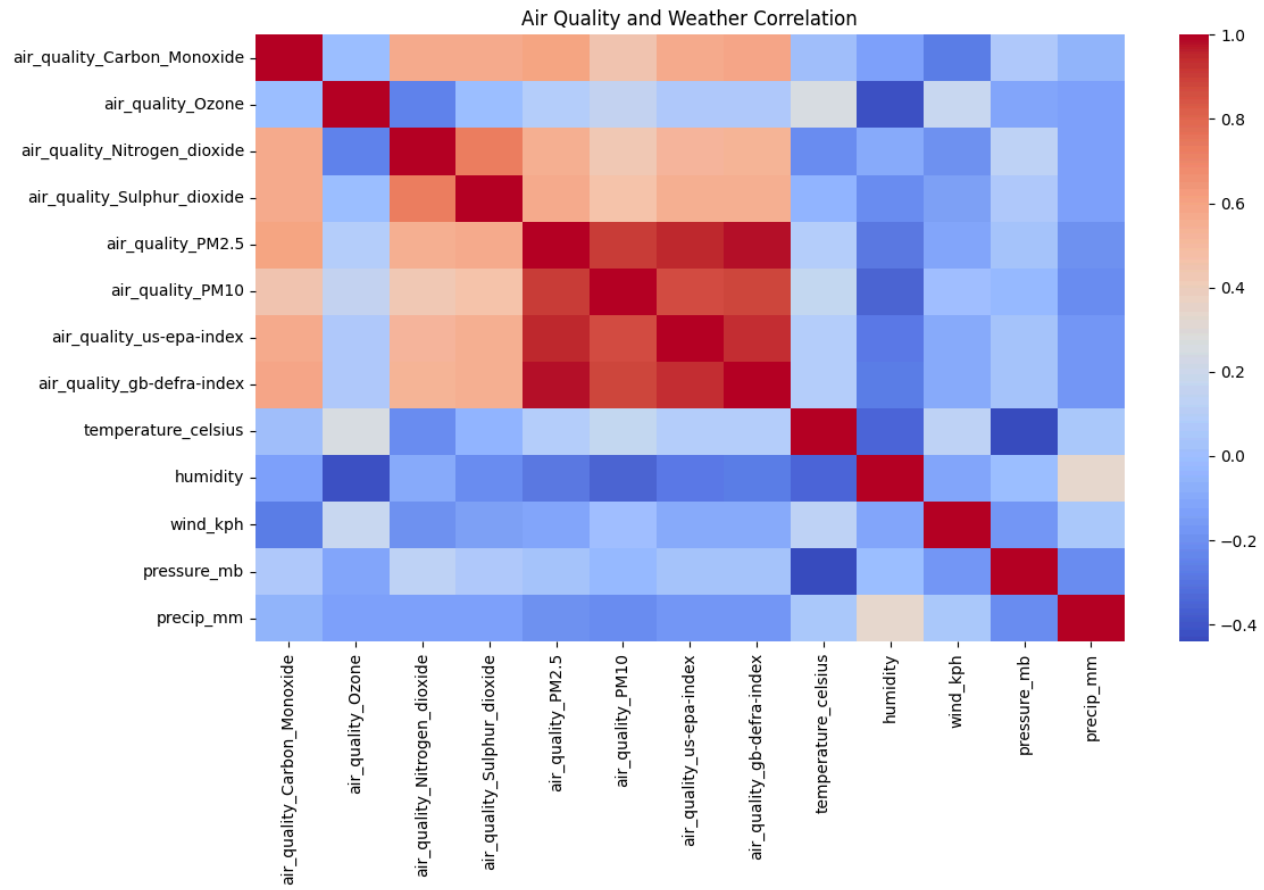
Advanced analysis — air quality correlation

```
air_quality_cols = [col for col in df.columns if "air_quality" in col.lower()]

air_quality_cols
```

```
['air_quality_Carbon_Monoxide',
 'air_quality_Ozone',
 'air_quality_Nitrogen_dioxide',
 'air_quality_Sulphur_dioxide',
 'air_quality_PM2.5',
 'air_quality_PM10',
 'air_quality_us-epa-index',
 'air_quality_gb-defra-index']
```

```
if len(air_quality_cols) > 0:
    selected_cols = air_quality_cols + [
        "temperature_celsius",
        "humidity",
        "wind_kph",
        "pressure_mb",
        "precip_mm"
    ]
```

```
    selected_cols = [col for col in selected_cols if col in df.columns]

    air_corr = df[selected_cols].corr()

    plt.figure(figsize=(12, 8))
    sns.heatmap(air_corr, annot=False, cmap="coolwarm")
    plt.title("Air Quality and Weather Correlation")
    plt.tight_layout()
    plt.savefig(PROJECT_PATH + "/outputs/figures/air_quality_correlation.png")
    plt.show()
else:
    print("No air quality columns found.")
```



Advanced analysis — spatial visualization

```
if "latitude" in df.columns and "longitude" in df.columns:
    fig = px.scatter_geo(
        df,
        lat="latitude",
        lon="longitude",
        color="temperature_celsius",
        hover_name="location_name" if "location_name" in df.columns else None,
        title="Global Temperature Distribution"
    )

    fig.show()

    fig.write_html(PROJECT_PATH + "/outputs/figures/global_temperature_map.html")
else:
    print("Latitude and longitude columns not found.")
```

Global Temperature Distribution